Dokumentasi Fitur Cetak PDF (Invoice, Kwitansi, Nota)

Oct 22, 2026 · @faridlan

Fitur PDF di Sikon Dashboard dibuat sepenuhnya di browser dengan jsPDF dan jspdf-autotable: PDF digambar lewat kode (bukan screenshot HTML), dipratinjau lewat iframe, dan diunduh dengan nama file otomatis.

Library dan metode

Dua library, keduanya berjalan di browser tanpa server PDF:

Library	Versi (package.json)	Fungsi
jspdf	^4.2.1 (terpasang 4.2.1)	Membuat dokumen PDF: teks, garis, kotak, gambar, watermark, ukuran/orientasi halaman
jspdf-autotable	^5.0.8	Plugin jsPDF untuk tabel produk (header, kolom, zebra row, pindah halaman otomatis)
html2canvas	tidak ada di package.json	Hanya dipakai Surat Penawaran (`QuotationPdfDialog.tsx`), bukan invoice/kwitansi/nota

Metodenya: **programmatic drawing**. Setiap teks, kotak, dan gambar diletakkan dengan koordinat milimeter (`doc.text(teks, x, y)`, `doc.rect(...)`, `doc.addImage(...)`), sehingga hasilnya PDF vektor yang tajam, teksnya bisa diseleksi, dan ukuran filenya kecil. Ini berbeda dengan metode html2canvas yang memotret elemen HTML menjadi gambar lalu menempelkannya ke PDF.

Stack aplikasinya: React 19, TanStack Start/Router, TanStack Query, shadcn/ui (Dialog, Button), dan sonner untuk toast. Semuanya hanya pembungkus UI; logika PDF-nya sendiri hanya butuh jsPDF dan autotable.

Alur kerja

Tiga dokumen memakai pola yang sama: satu fungsi generator murni di `src/lib/invoice.ts`, dan satu dialog React di `src/routes/orders.$orderId.tsx` yang memanggilnya.

1. Pengguna klik tombol di halaman detail order, lalu state dialog (`pdfOpen`, `notaOpen`, `kwitansiOpen`) menjadi `true`.
2. Saat dialog terbuka, `useEffect` memanggil generator (`generateInvoicePDF` atau `generateKwitansiPDF`) dengan data order, item, customer, pembayaran, dan opsi (stempel, tanda tangan, catatan).
3. Generator mengembalikan objek `jsPDF`. Dialog memanggil `doc.output("blob")`, membuat `URL.createObjectURL(blob)`, lalu menampilkannya di `<iframe src=...>` sebagai preview.
4. Setiap kali opsi berubah (centang stempel, ubah catatan, ganti rekening), `useEffect` berjalan ulang dan preview dibuat ulang. Cleanup effect memanggil `URL.revokeObjectURL` agar memori tidak bocor.
5. Klik **Download** memanggil generator sekali lagi, menyusun nama file, lalu `doc.save(fileName)` memicu unduhan.

Tiga pintu masuknya:

Dokumen	Tombol	Tampil kapan	Komponen	Fungsi generator
Invoice	Cetak Invoice	Semua status order	dialog inline di `OrderDetailPage`	`generateInvoicePDF`
Nota	Nota PDF	Hanya jika `payment_status === "paid"`	`NotaPdfDialog`	`generateInvoicePDF` dengan `isNota: true`
Kwitansi	Ikon printer di baris pembayaran	Per pembayaran	`KwitansiPdfDialog`	`generateKwitansiPDF`

Struktur generator PDF

`generateInvoicePDF` dan `generateKwitansiPDF` sama-sama `async` dan mengembalikan `Promise<jsPDF>`. Fungsinya tidak menyentuh DOM selain memuat gambar, jadi mudah dipindah ke project lain. Nota bukan fungsi terpisah: ia adalah invoice dengan flag

`options.isNota = true`.

Invoice A4 portrait (`new jsPDF()`, margin 20 mm) digambar berurutan dari atas ke bawah:

Bagian	Cara menggambar
Header	`doc.setFillColor(30,41,59)` + `doc.rect(0,0,pageWidth,50,"F")`, logo lewat `addImage`, nama perusahaan, alamat rata kanan, judul dokumen
Info tagihan	Nama customer (dipotong `splitTextToSize`), nomor (`INV-` atau `NOTA-` + nomor order), tanggal, status
Tabel item	`autoTable(doc, { head, body, columnStyles, alternateRowStyles })`; kolom No, Produk (+ detail bahan/warna), Qty, Harga/Unit, Subtotal
Total	Posisi Y diambil dari `doc.lastAutoTable.finalY`, lalu baris Subtotal, DPP/PPN/PPh 22 (jika kena pajak), Ongkir, Total, Sudah Dibayar, Sisa Tagihan, dan terbilang
Info bank	Daftar rekening di kiri bawah tabel (disembunyikan untuk nota)
Tanda tangan	Teks "Hormat Kami", lalu stempel dan gambar tanda tangan PNG jika dicentang
Watermark	Jika sisa tagihan 0, teks "LUNAS" ukuran 120 hijau dengan opacity 0,15
Footer	Teks abu-abu di `pageHeight - 20`

Perbedaan tiga dokumen:

	Invoice	Nota	Kwitansi
Halaman	A4 portrait	A4 portrait	A4 landscape (`new jsPDF("landscape","mm","a4")`)
Judul	INVOICE	NOTA	KWITANSI

	Invoice	Nota	Kwitansi
Nomor	`INV-<order_number>`	`NOTA-<order_number>`	`payment.reference_number` atau 8 karakter id pembayaran
Tanggal	`approved_at`, fallback `created_at`	`created_at` order	`payment.created_at`
Isi utama	Tabel item + total	Sama dengan invoice, tanpa info bank	Sudah terima dari, jumlah uang (terbilang), untuk pembayaran, kotak nominal
Data pembayaran	Semua pembayaran dijumlahkan	Sama	Satu pembayaran

Tiga teknik yang paling berguna untuk ditiru:

- **Posisi Y dinamis.** Karena tabel bisa panjang, semua elemen setelah tabel memakai `lastAutoTable.finalY` dan variabel `ty` yang ditambah tiap baris, bukan angka tetap.
- **Watermark transparan.** `doc.saveGraphicsState()`, `doc.setGState(new doc.GState({ opacity: 0.15 }))`, gambar teks, lalu `doc.restoreGraphicsState()`.
- **Gambar opsional.** Stempel dan tanda tangan hanya digambar jika flag `withStamp` / `withSignature` aktif, dibungkus `try/catch` agar gambar rusak tidak menggagalkan seluruh PDF.

Preview, download, nama file, dan print

Preview (view). PDF diubah menjadi Blob dan ditampilkan di iframe. Pola ini ada di ketiga dialog:

```
useEffect(() => {
  if (!open) return;
  let active = true;
  let url: string | null = null;

  (async () => {
    const doc = await generateInvoicePDF({ order, items, customer, payments,
bankAccounts, options });
    if (active) {
      url = URL.createObjectURL(doc.output("blob"));
```

```
      setPdfUrl(url);
    }
  })();

  return () => {
    active = false;
    if (url) URL.revokeObjectURL(url);   // bebaskan memori
  };
}, [open, order, items, customer, payments, options /* dependensi opsi */]);

// di JSX
<iframe src={pdfUrl} className="w-full h-[80vh]" />
```

Flag `active` mencegah hasil lama menimpa hasil baru bila pengguna mengubah opsi dengan cepat.

Download dengan nama file otomatis. Nama disusun dari jenis dokumen, nama customer (spasi diganti garis bawah), dan nomor order atau jenis pembayaran, lalu diberikan ke `doc.save()`:

```
const custName = (customer?.name || "Unknown").replace(/\s+/g, "_");

// Invoice
`Invoice-${custName}-${order.order_number ?? order.id.slice(0, 8)}.pdf`
// Nota
`Nota-${custName}-${order.order_number ?? order.id.slice(0, 8)}.pdf`
// Kwitansi
`Kwitansi-${custName}-${payment.payment_type}.pdf`

doc.save(fileName);   // jsPDF memicu unduhan di browser
```

Contoh hasil: `Invoice-PT_Maju_Jaya-ORD-0042.pdf`. Nama file yang sama muncul di jendela simpan browser tanpa pengguna mengetik apa pun.

Print. Kode invoice, nota, dan kwitansi tidak memanggil `window.print()`. Tombolnya berlabel "Cetak" karena membuka preview; pencetakan dilakukan lewat toolbar bawaan penampil PDF browser di dalam iframe (ikon printer), atau dari file yang sudah diunduh. Jika Anda ingin tombol Print sendiri, beri atribut id pdf-preview pada iframe dan tambahkan di dialog:

```
function handlePrint() {
  const iframe = document.querySelector<HTMLIFrameElement>("#pdf-preview");
  iframe?.contentWindow?.focus();
  iframe?.contentWindow?.print();
}
```

Alternatif lain tanpa iframe: `doc.autoPrint()` lalu `window.open(doc.output("bloburl"))`, yang membuka tab baru dan langsung menampilkan dialog print.

Dua dokumen lain di aplikasi ini (Surat Penawaran dan SPK/Surat Jalan di `src/lib/quotation.ts`) memakai metode berbeda: HTML dibangun sebagai string, dibuka di `window.open("", "_blank")`, lalu `window.print()` dipanggil otomatis setelah 300 ms. Itu bukan PDF jsPDF, tetapi jalur cetak lewat browser.

Fungsi pendukung

Semua ada di bagian atas `src/lib/invoice.ts` dan bisa disalin apa adanya.

Fungsi	Tugas	Catatan
`formatCurrency(n)`	`"Rp " + Math.round(n).toLocaleString("id-ID")`	Pemisah ribuan titik, tanpa desimal
`formatDate(s)`	`toLocaleDateString("id-ID", {day:"2-digit", month:"long", year:"numeric"})`	Hasil: `22 Oktober 2026`; tanggal kosong atau tidak valid menjadi `-`
`terbilang(n)`	Angka menjadi kata Indonesia, mis. `1.250.000` menjadi "Satu Juta Dua Ratus Lima Puluh Ribu Rupiah"	Rekursif sampai triliun; menangani "Seratus", "Seribu", "Sebelas"
`loadImageDataURL(url)`	`fetch` gambar, jadikan Blob, `FileReader.readAsDataURL`	Mengembalikan `null` jika gagal, jadi PDF tetap terbentuk tanpa logo
`formatOrderDetails(details)`	Mengubah JSON detail item (bahan, warna) menjadi teks multi-baris di sel produk	Spesifik ke domain seragam; ganti sesuai data Anda
`COMPANY`	Konstanta nama, alamat, telepon, email, website	Pindahkan ke konfigurasi atau env di project baru

Inti `terbilang`:

```
const satuan = ["", "Satu", "Dua", "Tiga", "Empat", "Lima", "Enam",
                "Tujuh", "Delapan", "Sembilan", "Sepuluh", "Sebelas"];

const convert = (v: number): string => {
  if (v < 12) return satuan[v];
  if (v < 20) return satuan[v - 10] + " Belas";
  if (v < 100) return satuan[Math.floor(v / 10)] + " Puluh" + (v % 10 ? " " +
convert(v % 10) : "");
  if (v < 200) return "Seratus" + (v % 100 ? " " + convert(v % 100) : "");
  if (v < 1000) return satuan[Math.floor(v / 100)] + " Ratus" + (v % 100 ? "
" + convert(v % 100) : "");
  if (v < 2000) return "Seribu" + (v % 1000 ? " " + convert(v % 1000) : "");
  if (v < 1e6) return convert(Math.floor(v / 1e3)) + " Ribu" + (v % 1e3 ? " "
+ convert(v % 1e3) : "");
  // lanjut dengan pola yang sama: Juta (1e6), Miliar (1e9), Triliun (1e12)
};

export const terbilang = (n: number) => convert(Math.round(n)) + " Rupiah";
```

Gambar (logo, stempel, tanda tangan) diambil dari folder `public/assets/` (`logo.png`, `stempel-wift.png`, `ttd-manager.png`) lewat URL relatif `/assets/...`. Gambar harus satu origin atau mengizinkan CORS agar `fetch` berhasil.

Template minimal untuk project lain

Instal dependensi:

```
npm i jspdf jspdf-autotable
```

1. Generator (`src/lib/pdf/invoice.ts`). Kerangka ini mengikuti struktur dan API jsPDF yang dipakai kode asli, dengan data diganti tipe generik. Template ini disusun dari kode asli dan belum dijalankan (npm tidak bisa diakses dari lingkungan saya), jadi uji dulu di project Anda:

```
import jsPDF from "jspdf";
import autoTable from "jspdf-autotable";

export interface PdfOptions { withStamp?: boolean; withSignature?: boolean;
note?: string }
interface Item { name: string; qty: number; price: number }
interface InvoiceInput {
```

```
  number: string; date: string; customerName: string;
  items: Item[]; paid: number; options?: PdfOptions;
}

const rp = (n: number) => "Rp " + Math.round(n || 0).toLocaleString("id-ID");

async function loadImage(url: string): Promise<string | null> {
  try {
    const res = await fetch(url);
    if (!res.ok) return null;
    const blob = await res.blob();
    return await new Promise((ok, fail) => {
      const r = new FileReader();
      r.onload = () => ok(r.result as string);
      r.onerror = fail;
      r.readAsDataURL(blob);
    });
  } catch { return null; }
}

export async function generateInvoicePDF(d: InvoiceInput): Promise<jsPDF> {
  const doc = new jsPDF();                        // A4 portrait, mm
  const W = doc.internal.pageSize.getWidth();
  const H = doc.internal.pageSize.getHeight();
  const M = 20;

  // Header
  doc.setFillColor(30, 41, 59);
  doc.rect(0, 0, W, 50, "F");
  const logo = await loadImage("/assets/logo.png");
  if (logo) { try { doc.addImage(logo, "PNG", M - 5, 12, 25, 25, undefined,
"FAST"); } catch {} }
  doc.setTextColor(255).setFontSize(22).setFont("helvetica", "bold");
  doc.text("NAMA PERUSAHAAN", M + 25, 24);
  doc.setFontSize(28).text("INVOICE", W - M, 46, { align: "right" });

  // Info
  doc.setTextColor(30, 41, 59).setFontSize(10);
  doc.text(`No. Invoice : ${d.number}`, M, 65);
  doc.text(`Tanggal     : ${d.date}`, M, 72);
  doc.text(`Kepada      : ${d.customerName}`, M, 79);

  // Tabel
```

```
  autoTable(doc, {
    startY: 90,
    head: [["No", "Produk", "Qty", "Harga", "Subtotal"]],
    body: d.items.map((it, i) => [i + 1, it.name, it.qty, rp(it.price),
rp(it.qty * it.price)]),
    margin: { left: M, right: M },
    styles: { fontSize: 9, cellPadding: 5 },
    headStyles: { fillColor: [30, 41, 59], textColor: 255 },
    columnStyles: { 0: { cellWidth: 15, halign: "center" }, 2: { halign:
"center" },
                    3: { halign: "right" }, 4: { halign: "right" } },
    alternateRowStyles: { fillColor: [245, 247, 250] },
  });

  // Total (Y mengikuti akhir tabel)
  const total = d.items.reduce((s, i) => s + i.qty * i.price, 0);
  const sisa = Math.max(0, total - d.paid);
  let y = (doc as any).lastAutoTable.finalY + 10;
  doc.setFont("helvetica", "normal").setFontSize(9);
  doc.text("Total", W - M - 85, y);        doc.text(rp(total),  W - M, y, {
align: "right" });
  y += 7;
  doc.text("Sudah Dibayar", W - M - 85, y); doc.text(rp(d.paid), W - M, y, {
align: "right" });
  y += 8;
  doc.setFont("helvetica", "bold").setFontSize(11);
  doc.text("Sisa Tagihan", W - M - 85, y);  doc.text(rp(sisa),   W - M, y, {
align: "right" });

  // Watermark LUNAS
  if (sisa <= 0) {
    doc.saveGraphicsState();
    doc.setGState(new (doc as any).GState({ opacity: 0.15 }));
    doc.setFontSize(120).setTextColor(34, 197, 94);
    doc.text("LUNAS", W / 2, H / 2, { align: "center" });
    doc.restoreGraphicsState();
  }
  return doc;
}
```

2. Dialog preview + download + print (React):

```
export function InvoicePdfDialog({ data, open, onClose }: Props) {
  const [url, setUrl] = useState<string | null>(null);
  const iframeRef = useRef<HTMLIFrameElement>(null);

  useEffect(() => {
    if (!open) return;
    let active = true, objUrl: string | null = null;
    generateInvoicePDF(data).then((doc) => {
      if (!active) return;
      objUrl = URL.createObjectURL(doc.output("blob"));
      setUrl(objUrl);
    }).catch(() => toast.error("Gagal membuat preview PDF"));
    return () => { active = false; if (objUrl) URL.revokeObjectURL(objUrl);
};
  }, [open, data]);

  async function download() {
    const doc = await generateInvoicePDF(data);
    const name = data.customerName.replace(/\s+/g, "_");
    doc.save(`Invoice-${name}-${data.number}.pdf`);
  }

  function print() {
    iframeRef.current?.contentWindow?.focus();
    iframeRef.current?.contentWindow?.print();
  }

  return (
    <Dialog open={open} onOpenChange={(v) => !v && onClose()}>
      <DialogContent className="max-w-5xl">
        <Button onClick={download} disabled={!url}>Download</Button>
        <Button variant="outline" onClick={print} disabled=
{!url}>Print</Button>
        {url ? <iframe ref={iframeRef} src={url} className="w-full h-[80vh]"
/> : <p>Membuat preview...</p>}
      </DialogContent>
    </Dialog>
  );
}
```

Untuk kwitansi, gunakan `new jsPDF("landscape", "mm", "a4")` dan susun label serta nilai dengan `doc.text(label, M, y)` dan `doc.text(": " + nilai, M + 40, y)`, naikkan `y` per baris. Untuk nota, pakai generator invoice yang sama dengan flag yang mengganti judul, awalan nomor, dan menyembunyikan blok rekening bank.

Checklist adopsi dan hal yang perlu diwaspadai

Langkah adopsi:

- [ ] Instal `jspdf` dan `jspdf-autotable`.
- [ ] Salin helper (`formatCurrency`, `formatDate`, `terbilang`, `loadImageDataURL`) ke file util.
- [ ] Ganti objek `COMPANY` dengan data perusahaan project baru (idealnya dari konfigurasi).
- [ ] Taruh logo, stempel, dan tanda tangan PNG di folder aset publik; sesuaikan path di `loadImageDataURL(...)`.
- [ ] Petakan tipe data order, item, customer, dan pembayaran project baru ke input generator.
- [ ] Buat satu dialog per dokumen dengan pola `useEffect` + Blob URL + iframe.
- [ ] Susun nama file otomatis di handler download, lalu tambahkan tombol Print bila diperlukan.
- [ ] Uji dengan item panjang (lebih dari satu halaman), customer bernama panjang, dan order lunas (watermark).

Hal yang saya temukan di kode asli dan sebaiknya tidak ikut tersalin begitu saja:

Temuan	Lokasi	Saran
Baris "Address:" dan "Phone:" customer tercetak tanpa nilai karena kode yang mengisinya dikomentari	`generateInvoicePDF`, bagian info tagihan	Isi dengan `customer.address` dan `customer.phone` di project baru
Total "Sudah Dibayar" di invoice menjumlahkan semua pembayaran tanpa melihat status, sedangkan halaman order dan kwitansi hanya menghitung yang `verified`	`generateInvoicePDF` memakai `payments.reduce(...)`; `OrderDetailPage` memakai filter `verified`	Filter status sebelum dijumlahkan agar angka PDF sama dengan layar

Temuan	Lokasi	Saran
Fallback DPP memakai `subtotal / 1.09` padahal tarif PPN default 12%	Perhitungan pajak di invoice dan di `OrderDetailPage`	Tentukan rumus pajak sesuai aturan bisnis project baru
Judul `isProforma ? "INVOICE" : "INVOICE"` sama di kedua cabang	Bagian header	Sederhanakan, atau bedakan menjadi "PROFORMA INVOICE"
Nama penandatangan dan kota "Tasikmalaya" ditulis langsung di kode	Bagian tanda tangan, kwitansi	Jadikan parameter
`html2canvas` diimpor oleh `QuotationPdfDialog.tsx` tetapi tidak tercantum di `package.json` (hanya ikut terpasang sebagai dependensi opsional jsPDF)	`package.json`	Jika memakai metode HTML ke PDF, deklarasikan `html2canvas` secara eksplisit

Keterbatasan teknis:

- Font bawaan `helvetica` hanya mendukung karakter Latin. Untuk aksara lain atau simbol khusus, embed font TTF dengan `doc.addFileToVFS` dan `doc.addFont`.
- Preview PDF di iframe tidak selalu tampil di browser mobile (terutama iOS Safari). Siapkan fallback berupa tombol Download atau `window.open(doc.output("bloburl"))`.
- Generator dipanggil dua kali (preview dan download) dan memuat gambar ulang tiap kali. Untuk dokumen banyak, simpan data URL gambar di cache modul.
- Karena PDF dibuat di browser, jangan pakai pendekatan ini untuk dokumen yang harus tidak bisa dimanipulasi pengguna (misalnya invoice resmi yang harus ditandatangani server). Untuk itu, buat PDF di server.